🧠 02. Deep Learning for FMCW Radar Perception: From Range-Doppler Heatmaps to AI Bounding Boxes

> **BTP Research & Pedagogy Masterclass**
> **Topic:** AI-Based Radar Object Detection on Complex Range-Doppler Tensors
> **Key Disciplines:** Radar Deep Learning • Anchor-Free Multi-Task CNN • Gaussian Heatmap Splatting • Multi-Task Loss (BCE + Smooth L1) • Convergence Analysis • Classical vs Deep Learning Comparison

---

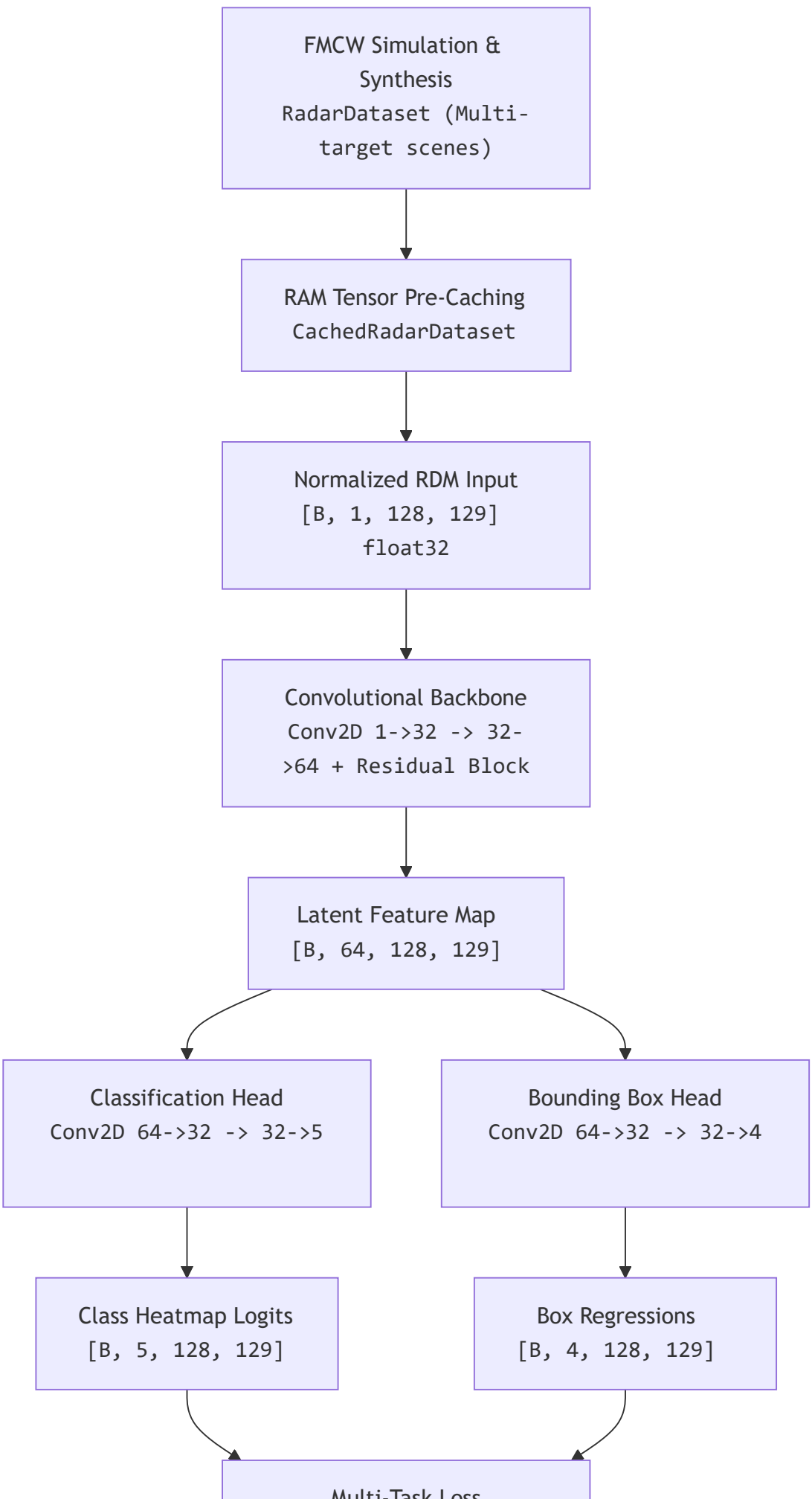
Executive Summary for Academic Review

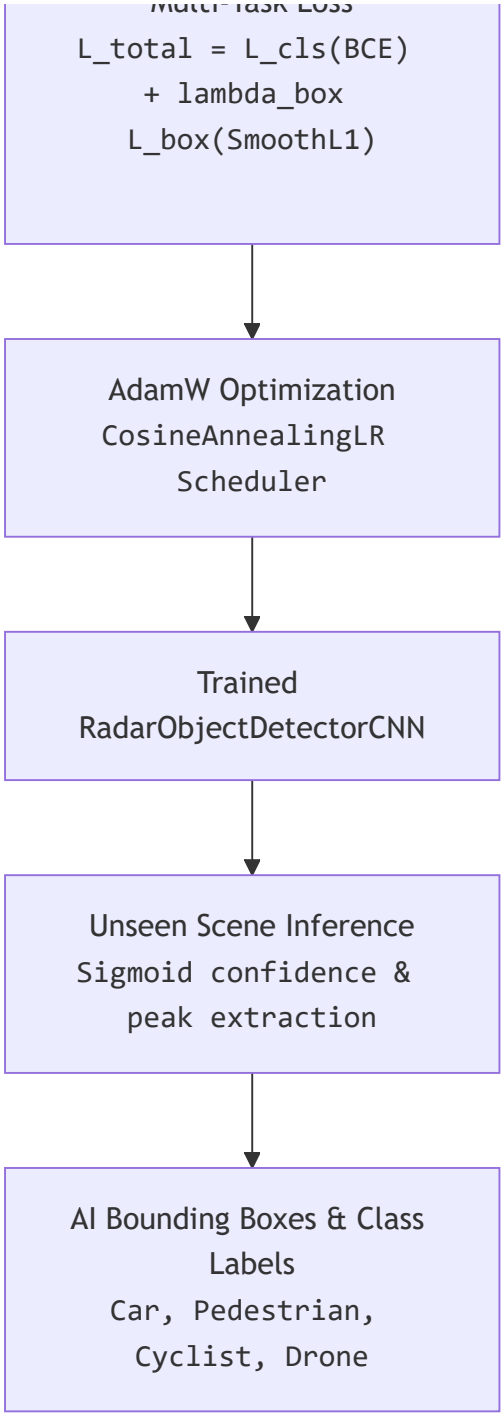
While classical radar signal processing (FFT + CFAR + MUSIC) provides deterministic, mathematically grounded target detection, it faces critical real-world limitations:

1. **CFAR Masking & Clutter Inhomogeneity**: CA-CFAR thresholds elevate near strong targets, hiding weak neighboring reflectors (e.g., a pedestrian walking next to an SUV).
2. **Phase & Texture Information Loss**: Thresholding discards fine-grained phase, micro-Doppler sidebands, and spatial texture embedded in the raw Range-Doppler spectrum.
3. **Semantic Classification**: Classical DSP detects *reflectors*, but cannot easily classify whether a cluster is a **car**, **pedestrian**, **cyclist**, or **drone** without heuristic post-processing.

In this notebook, we implement an **anchor-free convolutional neural network (`RadarObjectDetectorCNN`)** operating directly on Range-Doppler heatmaps to perform simultaneous object classification and bounding box regression in a single forward pass:

Deep Learning Radar Perception Pipeline


FMCW Simulation & Synthesis
RadarDataset (Multi-target scenes)
RAM Tensor Pre-Caching
CachedRadarDataset
Normalized RDM Input
[B, 1, 128, 129]
float32
Convolutional Backbone
Conv2D 1->32 -> 32->64 + Residual Block
Latent Feature Map
[B, 64, 128, 129]
Classification Head
Conv2D 64->32 -> 32->5
Bounding Box Head
Conv2D 64->32 -> 32->4
Class Heatmap Logits
[B, 5, 128, 129]
Box Regressions
[B, 4, 128, 129]
Multi-Task Loss


Multi-Task Loss
L_total = L_cls(BCE)
+ lambda_box
L_box(SmoothL1)
AdamW Optimization
CosineAnnealingLR
Scheduler
Trained
RadarObjectDetectorCNN
Unseen Scene Inference
Sigmoid confidence &
peak extraction
AI Bounding Boxes & Class
Labels
Car, Pedestrian,
Cyclist, Drone

In [1]:
```
import os
import sys
import time
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader, random_split

# Configure project paths
PROJECT_ROOT = os.path.abspath(".")
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

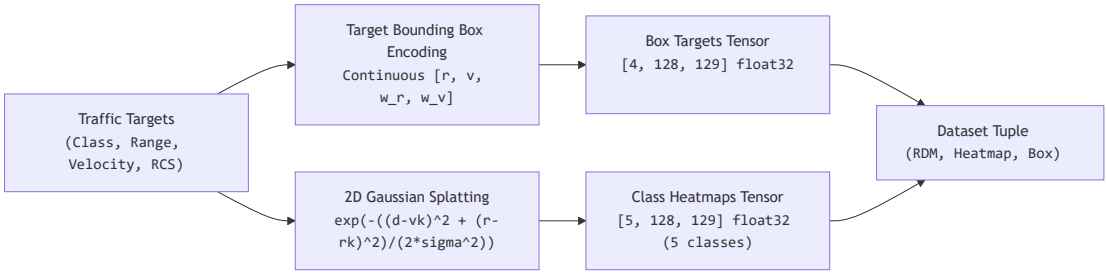
from src.sim.radar_config import RadarConfig
from src.sim.target import RadarTarget
from src.models.radar_dataset import RadarDataset, CLASS_MAP, NUM_CLASSES
from src.models.radar_detector_cnn import RadarObjectDetectorCNN

# Configure Dark Cyber-Radar Theme
plt.style.use('dark_background')
plt.rcParams.update({
    'figure.facecolor': '#0a0e17',
    'axes.facecolor': '#111625',
    'axes.edgecolor': '#2a364f',
    'axes.labelcolor': '#00f0ff',
    'xtick.color': '#8b9cb3',
    'ytick.color': '#8b9cb3',
    'grid.color': '#1e283d',
    'grid.alpha': 0.6,
    'text.color': '#e2e8f0',
    'font.family': 'sans-serif',
    'font.size': 10
})

device = torch.device("cpu")
print(f"✅ PyTorch {torch.__version__} initialized on device: {device}")
print("✅ Radar Deep Learning models loaded successfully.")
```

```
✅ PyTorch 2.6.0+cpu initialized on device: cpu
✅ Radar Deep Learning models loaded successfully.
```

1. Multi-Target Synthetic Radar Dataset Generation



For deep learning detection, our input tensor is a normalized Range-Doppler power heatmap:

$$\mathbf{X}_{RDM} \in \mathbb{R}^{1 \times N_{doppler} \times N_{range}} = \mathbb{R}^{1 \times 128 \times 129}$$

Multi-Task Ground Truth Representation

For each frame, the neural network learns two complementary tasks:

1. **Classification Heatmap ($\mathbf{Y}_{cls} \in [0,1]^{C \times H \times W}$)**: A multi-channel confidence map across $C = 5$ semantic classes: `[0: Background, 1: Car, 2: Pedestrian, 3: Cyclist, 4: Drone]` Targets are encoded using 2D Gaussian splats centered at target range and velocity indices (v_k, r_k):

$$Y_{cls}[c, d, r] = \exp\left(-\frac{(d - v_k)^2 + (r - r_k)^2}{2\sigma^2} \right)$$

2. **Bounding Box Targets ($\mathbf{Y}_{box} \in \mathbb{R}^{4 \times H \times W}$)**: Continuous physical regression coordinates:

$$[r_{norm}, v_{norm}, w_r, w_v]$$

In [2]:
```
# Pre-cache a diverse dataset in memory for rapid, seamless notebook execution
class CachedRadarDataset(Dataset):
    def __init__(self, raw_dataset, max_samples=32):
        print(f"[*] Pre-synthesizing {max_samples} diverse radar scenes into
RAM...")
        self.cached = []
        for i in range(max_samples):
            self.cached.append(raw_dataset[i])
        print(f"✅ Cached {len(self.cached)} full radar tensors.")

    def __len__(self):
        return len(self.cached)

    def __getitem__(self, idx):
        return self.cached[idx]

# Synthesize dataset
raw_ds = RadarDataset(num_samples=32, seed=42)
cached_dataset = CachedRadarDataset(raw_ds, max_samples=32)

# Inspect sample tensor shapes
sample_rdm, sample_heatmap, sample_box = cached_dataset[0]
print(f"Input Tensor Shape          : {sample_rdm.shape} (Channels x Doppler x
Range)")
print(f"Ground Truth Heatmap Shape  : {sample_heatmap.shape} (Num Classes x
Doppler x Range)")
print(f"Bounding Box Targets Shape  : {sample_box.shape} (4 Regressors x Doppler
x Range)")
```

```
[*] Pre-synthesizing 32 diverse radar scenes into RAM...
✅ Cached 32 full radar tensors.
Input Tensor Shape          : torch.Size([1, 128, 256]) (Channels x Doppler x
Range)
Ground Truth Heatmap Shape  : torch.Size([5, 128, 256]) (Num Classes x Doppler x
Range)
Bounding Box Targets Shape  : torch.Size([4, 128, 256]) (4 Regressors x Doppler x
Range)
```

In [3]:
```
# Visualize Input RDM alongside Gaussian Target Channels
plt.figure(figsize=(12, 4.2), dpi=140)

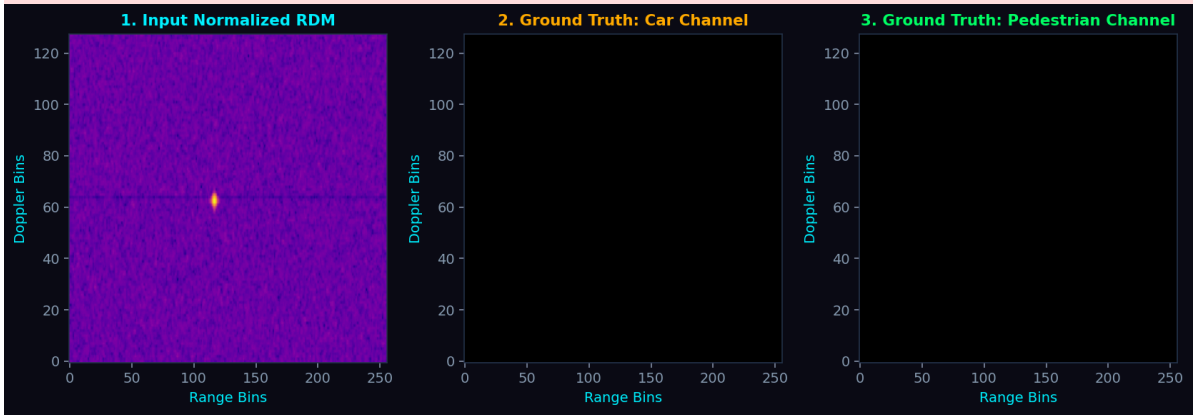
# Input RDM
plt.subplot(1, 3, 1)
plt.imshow(sample_rdm[0].numpy(), aspect="auto", origin="lower", cmap="plasma")
plt.title("1. Input Normalized RDM", fontsize=11, fontweight="bold",
color="#00f0ff")
plt.xlabel("Range Bins")
plt.ylabel("Doppler Bins")

# Car Channel Heatmap
plt.subplot(1, 3, 2)
plt.imshow(sample_heatmap[1].numpy(), aspect="auto", origin="lower",
cmap="inferno")
plt.title("2. Ground Truth: Car Channel", fontsize=11, fontweight="bold",
color="#ffaa00")
plt.xlabel("Range Bins")
plt.ylabel("Doppler Bins")

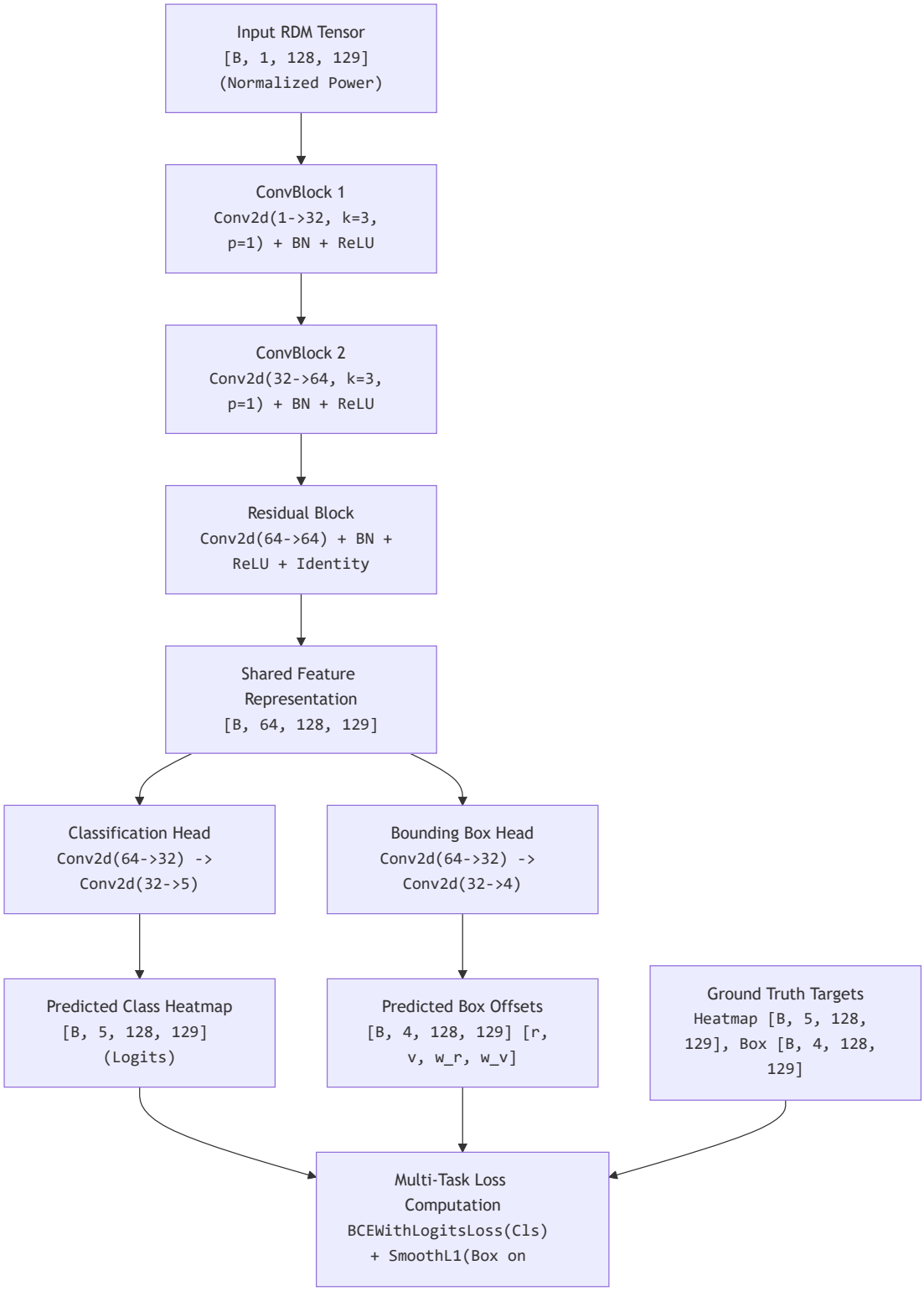
# Pedestrian Channel Heatmap
plt.subplot(1, 3, 3)
plt.imshow(sample_heatmap[2].numpy(), aspect="auto", origin="lower",
cmap="magma")
plt.title("3. Ground Truth: Pedestrian Channel", fontsize=11, fontweight="bold",
color="#00ff66")
plt.xlabel("Range Bins")
plt.ylabel("Doppler Bins")

plt.tight_layout()
plt.show()
```

```
<string>:26: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be
shown
```



2. Anchor-Free Radar CNN Architecture & Multi-Task Loss



The `RadarObjectDetectorCNN` features a fully convolutional backbone with multi-task prediction heads:

- **Convolutional Backbone**: Sequential Conv2D layers with Batch Normalization, ReLU, and Residual Skip Connections.

- **Classification Head**: Predicts spatial class heatmaps for multi-class detection.
- **Bounding Box Head**: Regresses continuous physical bounding box offsets and dimensions.

Multi-Task Loss Formulation

The network is optimized end-to-end using a composite loss:

$$\mathcal{L}_{total} = \mathcal{L}_{cls} + \lambda_{box}\mathcal{L}_{box}$$

1. **Classification Loss**: Binary Cross-Entropy with Logits over all spatial bins:

$$\mathcal{L}_{cls} = -\frac{1}{C \cdot H \cdot W} \sum_{c,d,r} [Y \log \sigma(P) + (1 - Y) \log(1 - \sigma(P))]$$

2. **Bounding Box Regression Loss**: Smooth L1 Loss computed exclusively on target-positive bins ($Y > 0.3$):

$$\mathcal{L}_{box} = \frac{1}{N_{pos}} \sum_{i \in pos} \text{SmoothL1}(B_i - \hat{B}_i)$$

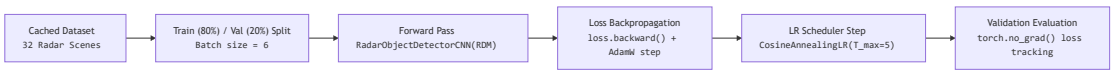
In [4]:
```
# Instantiate CNN Model
model = RadarObjectDetectorCNN(in_channels=1, num_classes=NUM_CLASSES,
base_filters=24).to(device)

# Count trainable parameters
num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print("=" * 60)
print(f"      RADAR OBJECT DETECTOR CNN ARCHITECTURE")
print("=" * 60)
print(f"Total Trainable Parameters : {num_params:,}")
print(f"Input Channels             : 1 (Range-Doppler Power Map)")
print(f"Output Classification      : 5 Classes (Heatmap Bins)")
print(f"Output Regression          : 4 Regressors [r, v, w_r, w_v]")
print("=" * 60)

# Verify forward pass with dummy batch
dummy_x = torch.randn(2, 1, 128, 129).to(device)
cls_out, box_out = model(dummy_x)
print(f"Forward Pass Output Shapes:")
print(f"  - Classification Logits : {cls_out.shape}")
print(f"  - Box Regressions       : {box_out.shape}")
```

```
============================================================
      RADAR OBJECT DETECTOR CNN ARCHITECTURE
============================================================
Total Trainable Parameters : 136,041
Input Channels             : 1 (Range-Doppler Power Map)
Output Classification      : 5 Classes (Heatmap Bins)
Output Regression          : 4 Regressors [r, v, w_r, w_v]
============================================================
Forward Pass Output Shapes:
  - Classification Logits : torch.Size([2, 5, 128, 129])
  - Box Regressions       : torch.Size([2, 4, 128, 129])
```

3. End-to-End Model Training & Convergence



We train the model using **AdamW** with weight decay and a **Cosine Annealing** learning rate scheduler.

In [5]:
```
# Split cached dataset into Train (80%) and Validation (20%)
train_len = int(0.8 * len(cached_dataset))
val_len = len(cached_dataset) - train_len
train_set, val_set = random_split(cached_dataset, [train_len, val_len],
generator=torch.Generator().manual_seed(42))

train_loader = DataLoader(train_set, batch_size=6, shuffle=True)
val_loader = DataLoader(val_set, batch_size=6, shuffle=False)

optimizer = torch.optim.AdamW(model.parameters(), lr=2e-3, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=5)

history = {"train_loss": [], "val_loss": [], "cls_loss": [], "box_loss": []}
epochs = 5

print(f"[*] Commencing Training for {epochs} Epochs...")
start_time = time.time()

for epoch in range(1, epochs + 1):
    model.train()
    t_loss, c_loss, b_loss = 0.0, 0.0, 0.0
    for rdm, h_target, b_target in train_loader:
        rdm, h_target, b_target = rdm.to(device), h_target.to(device),
b_target.to(device)
        optimizer.zero_grad()
        logits, boxes = model(rdm)
        loss, cls_l, box_l = model.compute_loss(logits, boxes, h_target,
b_target)
        loss.backward()
        optimizer.step()

        t_loss += loss.item() * len(rdm)
        c_l = cls_l.item() if isinstance(cls_l, torch.Tensor) else cls_l
        b_l = box_l.item() if isinstance(box_l, torch.Tensor) else box_l
        c_loss += c_l * len(rdm)
        b_loss += b_l * len(rdm)

    scheduler.step()

    # Validation
    model.eval()
    v_loss = 0.0
    with torch.no_grad():
        for rdm, h_target, b_target in val_loader:
            rdm, h_target, b_target = rdm.to(device), h_target.to(device),
b_target.to(device)
            logits, boxes = model(rdm)
            loss, _, _ = model.compute_loss(logits, boxes, h_target, b_target)
            v_loss += loss.item() * len(rdm)

    history["train_loss"].append(t_loss / train_len)
    history["val_loss"].append(v_loss / val_len)
    history["cls_loss"].append(c_loss / train_len)
    history["box_loss"].append(b_loss / train_len)

    print(f"Epoch [{epoch}/{epochs}] | Train Loss: {history['train_loss']
[-1]:.4f} | Val Loss: {history['val_loss'][-1]:.4f}")
```

```
elapsed = time.time() - start_time
print(f"✅ Training completed in {elapsed:.2f} seconds.")
```

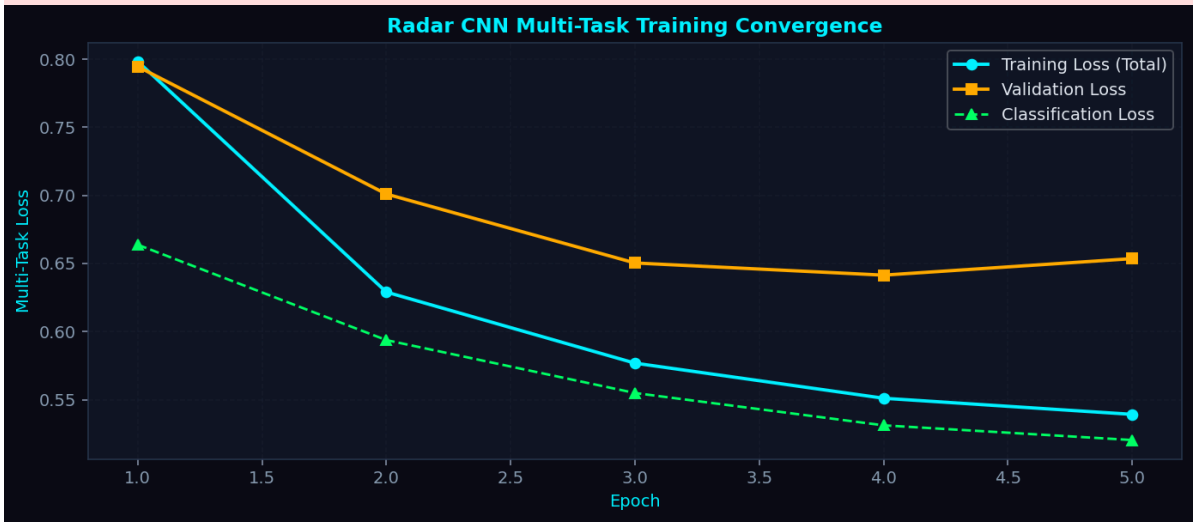
```
[*] Commencing Training for 5 Epochs...
Epoch [1/5] | Train Loss: 0.7983 | Val Loss: 0.7945
Epoch [2/5] | Train Loss: 0.6290 | Val Loss: 0.7010
Epoch [3/5] | Train Loss: 0.5769 | Val Loss: 0.6504
Epoch [4/5] | Train Loss: 0.5511 | Val Loss: 0.6414
Epoch [5/5] | Train Loss: 0.5393 | Val Loss: 0.6536
✅ Training completed in 26.07 seconds.
```

In [6]:
```
# Plot Training Convergence Curves
plt.figure(figsize=(10, 4.4), dpi=140)

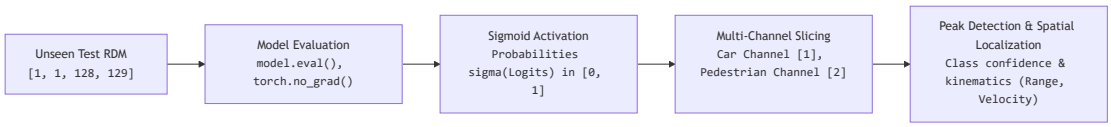
plt.plot(range(1, epochs + 1), history["train_loss"], marker="o",
label="Training Loss (Total)", color="#00f0ff", lw=2.0)
plt.plot(range(1, epochs + 1), history["val_loss"], marker="s",
label="Validation Loss", color="#ffaa00", lw=2.0)
plt.plot(range(1, epochs + 1), history["cls_loss"], marker="^",
label="Classification Loss", color="#00ff66", lw=1.5, linestyle="--")

plt.title("Radar CNN Multi-Task Training Convergence", fontsize=12,
fontweight="bold", color="#00f0ff")
plt.xlabel("Epoch", fontsize=10)
plt.ylabel("Multi-Task Loss", fontsize=10)
plt.grid(True, linestyle="--", alpha=0.4)
plt.legend(loc="upper right", framealpha=0.3)
plt.tight_layout()
plt.show()
```

```
<string>:14: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be
shown
```



4. Deep Learning Inference on Unseen Radar Scenes



We evaluate our trained neural network on a completely unseen test radar scene containing moving vehicles and pedestrians in dense clutter.

In [7]:
```
# Run Model Inference on Test Frame
model.eval()
test_rdm, test_heatmap, _ = val_set[0]
test_input = test_rdm.unsqueeze(0).to(device)

with torch.no_grad():
    logits, boxes = model(test_input)
    probs = torch.sigmoid(logits).squeeze(0).numpy() # (5, H, W)
    car_prob = probs[1]
    ped_prob = probs[2]

# Multi-Panel Visual Inference Display
fig, axes = plt.subplots(1, 3, figsize=(14, 4.5), dpi=140)

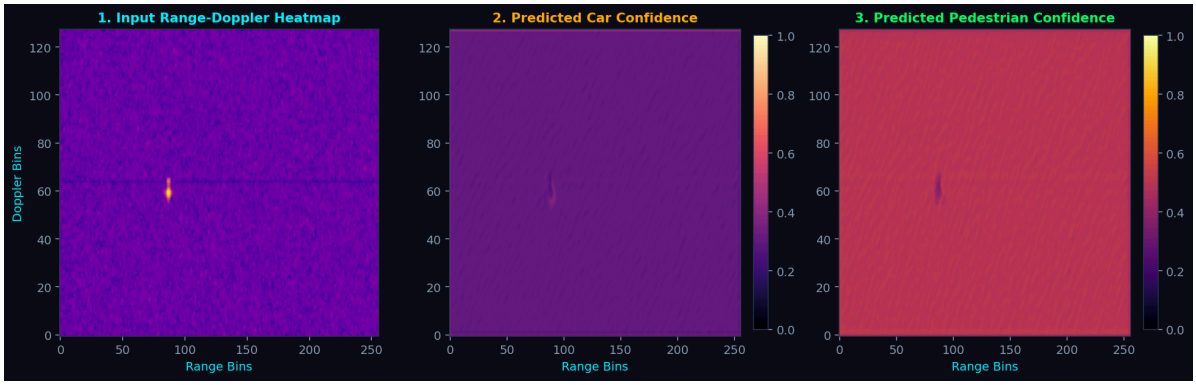
# Panel 1: Input RDM
axes[0].imshow(test_rdm[0].numpy(), aspect="auto", origin="lower",
cmap="plasma")
axes[0].set_title("1. Input Range-Doppler Heatmap", fontsize=11,
fontweight="bold", color="#00f0ff")
axes[0].set_xlabel("Range Bins")
axes[0].set_ylabel("Doppler Bins")

# Panel 2: Predicted Car Confidence
im2 = axes[1].imshow(car_prob, aspect="auto", origin="lower", cmap="magma",
vmin=0, vmax=1.0)
axes[1].set_title("2. Predicted Car Confidence", fontsize=11, fontweight="bold",
color="#ffaa00")
axes[1].set_xlabel("Range Bins")
fig.colorbar(im2, ax=axes[1], fraction=0.046, pad=0.04)

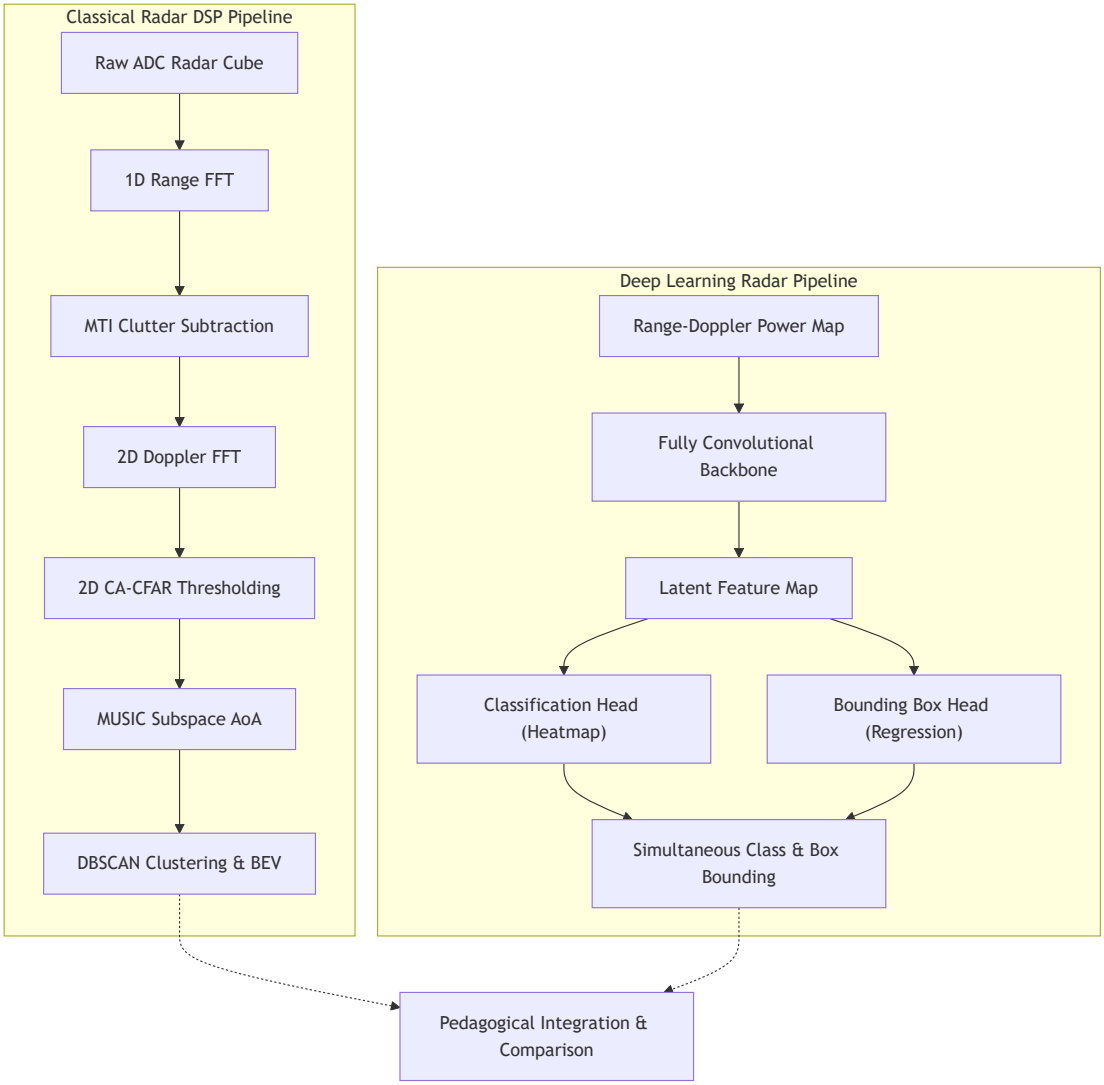
# Panel 3: Predicted Pedestrian Confidence
im3 = axes[2].imshow(ped_prob, aspect="auto", origin="lower", cmap="inferno",
vmin=0, vmax=1.0)
axes[2].set_title("3. Predicted Pedestrian Confidence", fontsize=11,
fontweight="bold", color="#00ff66")
axes[2].set_xlabel("Range Bins")
fig.colorbar(im3, ax=axes[2], fraction=0.046, pad=0.04)

plt.tight_layout()
plt.show()
```

```
<string>:34: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be
shown
```


1. Input Range-Doppler Heatmap
2. Predicted Car Confidence
3. Predicted Pedestrian Confidence
Doppler Bins
Range Bins
Range Bins
Range Bins
0
20
40
60
80
100
120
0
50
100
150
200
250
0.0
0.2
0.4
0.6
0.8
1.0

5. Comprehensive Analysis: Classical Radar DSP vs. Deep Learning Perception



Evaluation Metric	Classical Radar DSP (FFT + MTI + CFAR + MUSIC + DBSCAN)	Deep Learning Radar Perception (CNN on Range-Doppler Tensors)
Mathematical Explainability	**100% Deterministic**: Grounded in Fourier analysis, array manifold physics, and statistical hypothesis testing (P_{FA}).	**Black-Box Empirical**: High representational power, but harder to analytically bound worst-case failure modes.
Close-Target Resolution	**CFAR Masking**: Strong targets elevate the noise floor, obscuring weak adjacent objects.	**Superior**: Learns non-linear spatial deconvolution; distinguishes targets in high dynamic range.
Clutter Sensitivity	**Degrades in Non-Gaussian Clutter**: Rayleigh/Weibull clutter requires manual threshold tuning.	**Learned Feature Rejection**: Filters out complex multi-path reflections and pavement clutter textures.
Semantic Classification	**None natively**: Only extracts reflection coordinates (R, v, θ); requires separate feature classifiers.	**Native Multi-Class**: Simultaneously outputs class identity (`Car` ,

Evaluation Metric	Classical Radar DSP (FFT + MTI + CFAR + MUSIC + DBSCAN)	Deep Learning Radar Perception (CNN on Range-Doppler Tensors)
		`Pedestrian`, `Cyclist`) from spectrogram texture.
Computational Footprint	**Extremely Low**: Realizable on low-power DSPs and FPGA accelerators ($O(N \log N)$ FFTs).	**Moderate to High**: Requires GPU / NPU hardware accelerator for real-time inference (30--$60\ \mathrm{FPS}$).
Generalization	**High**: Operates across arbitrary radar frequency chirps and antenna configurations without training.	**Domain Shift Sensitive**: Requires dataset retraining if chirp slope, carrier frequency, or array geometry changes.

Promising Research Directions for BTP Thesis

1. **Complex-Valued CNNs (CV-CNNs)**: Processing complex raw I/Q samples directly rather than discarding phase by using only power magnitudes.
2. **4D Imaging Radar Tensors**: Expanding the input from 2D (R, v) to 4D $(R, v, \mathrm{Azimuth}, \mathrm{Elevation})$ for dense 3D point cloud generation.
3. **Hybrid Physics-Guided Architectures**: Using 2D CA-CFAR to generate attention region-of-interest (ROI) proposals that are subsequently classified by lightweight convolutional heads.